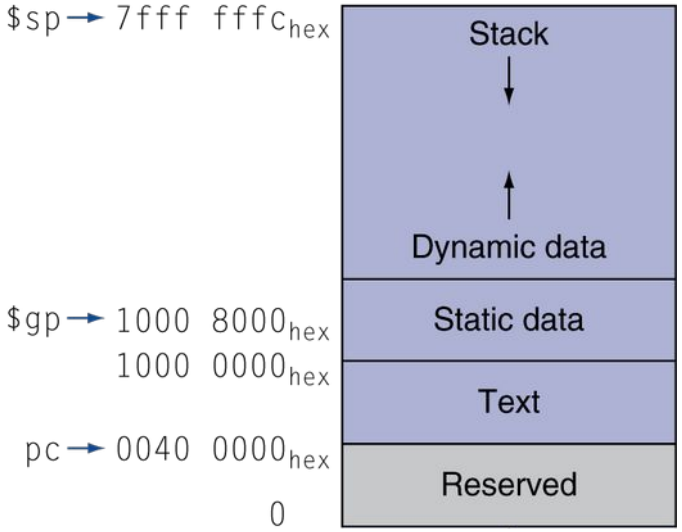
Chapter 2

Instructions: Language of the Computer

Memory Layout

- Text: program code
- Static data: global variables
  - e.g., static variables in C, constant arrays and strings
  - $gp initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage



Memory Layout

- Text: program code
- Static data: global

Memory Region	What is stored here?	Example
Text	Program instructions	add $t0, $t1, $t2
Static Data	Global / static data	Global variables, constant arrays, strings
Heap	Dynamically allocated data	malloc, new
Stack	Automatic storage	Local/temporary data*



- Dynamic data: heap
  - E.g., malloc in C, new in Java
- Stack: automatic storage

Character Data

- Byte-encoded character sets
  - ASCII: 128 characters
    - 95 graphic, 33 control
  - Latin-1: 256 characters
    - ASCII, +96 more graphic characters
- Unicode: 32-bit character set
  - Used in Java, C++ wide characters, …
  - Most of the world's alphabets, plus symbols
  - UTF-8, UTF-16: variable-length encodings

Character Data

- Byte-encoded character sets
  - ASCII: 128 characters

Encoding	Length	Simple Description
ASCII	1 byte (8 bits)	English letters, digits, symbols, control characters
Latin-1	1 byte (8 bits)	ASCII + additional Western European characters
UTF-8	1–4 bytes	Variable-length Unicode encoding
UTF-16	2 or 4 bytes	Variable-length Unicode encoding

  - UTF-8, UTF-16: variable-length encodings

Filling the Byte

- Could use bitwise operations
- MIPS byte/halfword load/store
  - String processing is a common case

```
lb rt, offset(rs)      lh rt, offset(rs)
```

  - Sign extend to 32 bits in rt

```
lbu rt, offset(rs)     lhu rt, offset(rs)
```

  - Zero extend to 32 bits in rt

```
sb rt, offset(rs)      sh rt, offset(rs)
```

  - Store just rightmost byte/halfword

Filling the Byte

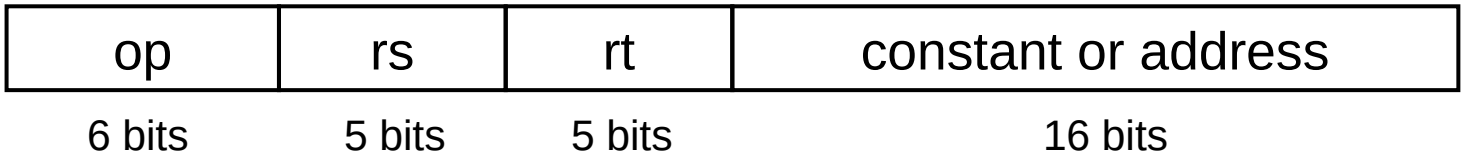
Data size	Load	Store
Word (32 bits)	lw	sw
Byte (8 bits)	lb / lbu	sb

- MIPS byte/halfword load/store

Q1. Assume:
$s0 = 0x1000
Memory[0x1000] = 1000 0001
$t0 = 00010010 00110100 01010110 10101011
Determine the result of each instruction independently:
(a) lb $t1, 0($s0)
(b) lbu $t1, 0($s0)
(c) sb $t0, 0($s0)
A1.
(a) $t1 = 11111111 11111111 11111111 10000001
(b) $t1 = 00000000 00000000 00000000 10000001
(c) Memory[0x1000] = 10101011

Branch Addressing

- Branch instructions specify
  - Opcode, two registers, target address
- Most branch targets are near branch
  - Forward or backward



- PC-relative addressing
  - Target address = PC + offset × 4
  - PC already incremented by 4 by this time

Branch Addressing

- Branch instructions specify

Q2. A MIPS branch instruction is located at memory address 1200.The 16-bit branch offset stored in the instruction is 5.
(a)What is the value of the PC used to calculate the branch target?
(b) What is the target address if the branch is taken?
(c) How many bytes away is the target from the next instruction?
A2.
(a)PC=1200+4 = 1204
(b)Target=1204+(5*4) = 1224
(c) 5*4 = 20 bytes

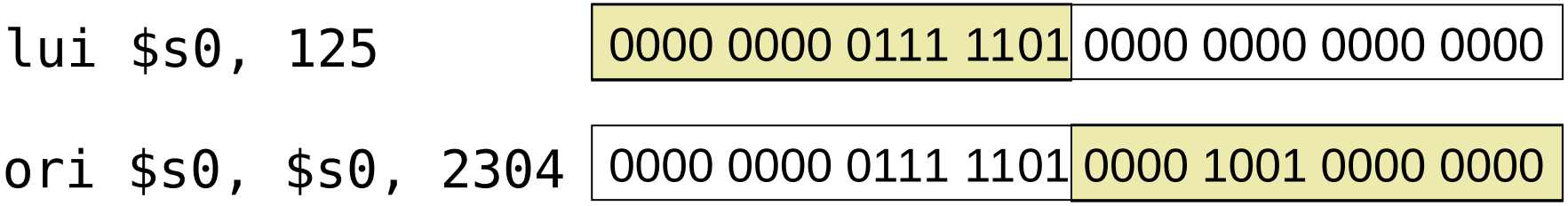
Target address = PC + offset × 4

- PC already incremented by 4 by this time

32-bit Constants

- Most constants are small
  - 16-bit immediate is sufficient
- For the occasional 32-bit constant

  `lui rt, constant`
  - Copies 16-bit constant to left 16 bits of rt
  - Clears right 16 bits of rt to 0



32-bit Constants

- Most constants are small

Q3. Load the following 32-bit constant into $s0 using lui and ori: 0000 0000 0011 0101 | 0000 0000 0001 0010

(a) What decimal immediate should be used for lui?

(b) What decimal immediate should be used for ori?

(c) Write the two MIPS instructions.

A3.

(a) 0000 0000 0011 0101 = 53

(b) 0000 0000 0001 0010 = 18

(c) lui $s0, 53

ori $s0, $s0, 18

`lui $s0, 61`	0000 0000 0011 1101	0000 0000 0000 0000
`ori $s0, $s0, 2304`	0000 0000 0111 1101	0000 1001 0000 0000

Target Addressing Example

- Loop code from earlier example
  - Assume Loop at location 80000

```
Loop: sll  $t1, $s3, 2      80000
      add  $t1, $t1, $s6    80004
      lw   $t0, 0($t1)      80008
      bne  $t0, $s5, Exit   80012
      addi $s3, $s3, 1      80016
      j    Loop             80020
Exit: …                     80024
```

0	0	19	9	4	0
0	9	22	9	0	32
35	9	8	0		
5	8	21	2		
8	19	19	1		
2	20000				

Q4.

(a)Identify the instruction format (R, I, or J) for each instruction.

(b) Identify the immediate or shift amount used in each applicable instruction.

(c) For bne and j, identify the value stored in their target/address field.

```
Loop: sll  $t1, $s3, 2      80000
      add  $t1, $t1, $s6    80004
      lw   $t0, 0($t1)      80008
      bne  $t0, $s5, Exit   80012
      addi $s3, $s3, 1      80016
      j    Loop             80020
Exit: …                     80024
```

0	0	19	9	4	0
0	9	22	9	0	32
35	9	8	0		
5	8	21	2		
8	19	19	1		
2	20000				

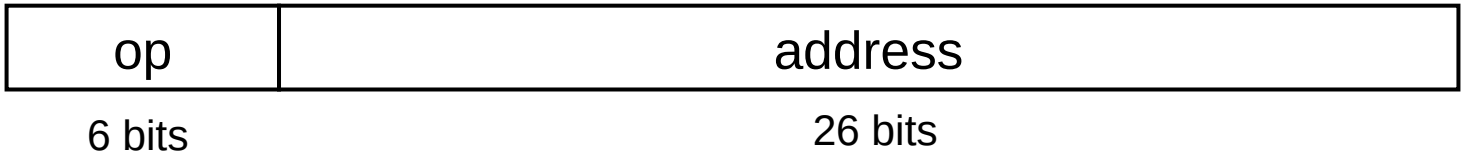
A4.

(a)R, R, I, I, I, J (b) sll: shamt=2, lw: i=0, addi: i=1

(c) bne: offset=2, j: address =20000

Jump Addressing

- Jump (`j` and `jal`) targets could be anywhere in text segment
  - Encode Encode target information using a 26-bit address field



- (Pseudo)Direct jump addressing
  - Target address = $PC_{31...28}$: (address × 4)

Jump Addressing

- Jump (j and jal) targets could be

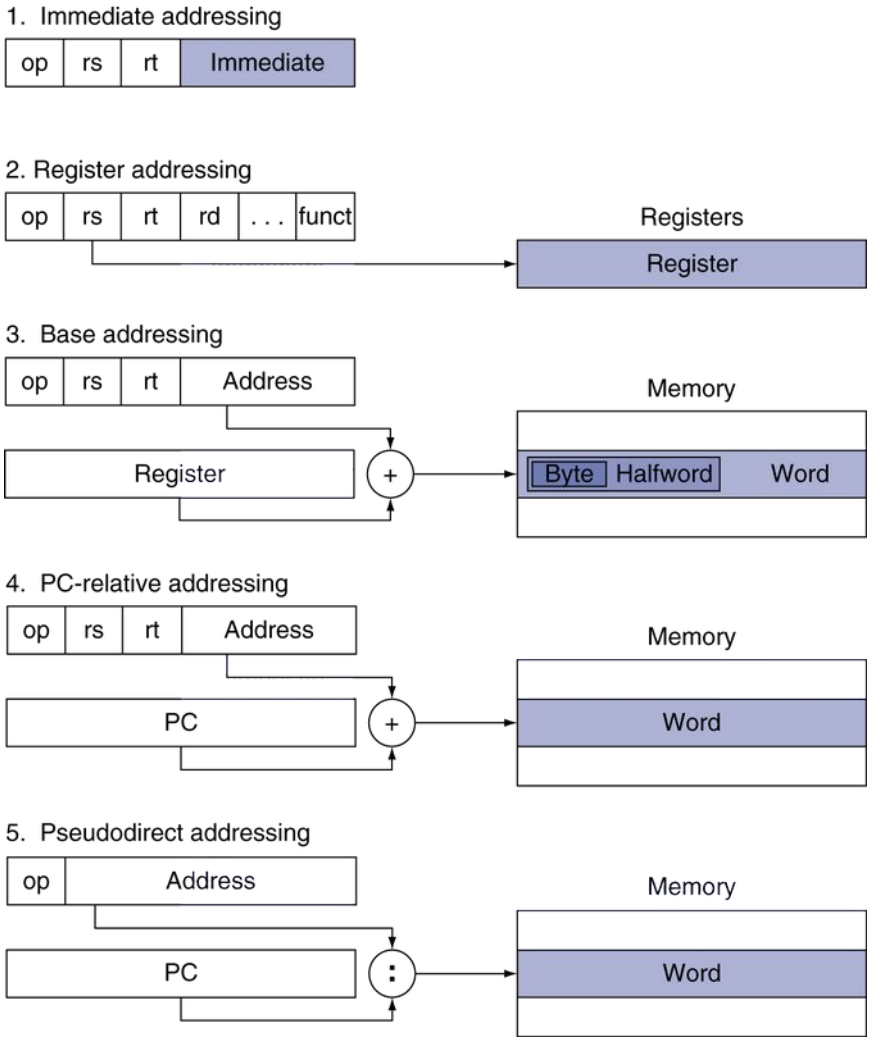
Q5. Suppose a MIPS j instruction needs to jump to the following 32-bit target address:
0010 0000 0000 0000 0000 0000 0010 1000
Assume the current PC has the same upper 4 bits (0010).
(a) Which bits are actually stored in the 26-bit address field of the J-format instruction?
(b) Explain why the upper 4 bits and lower 2 bits do not need to be stored in the instruction.
A5.
(a) 0010 0000 0000 0000 0000 0000 0010 1000.
(b) Upper 4 bits come from PC[31:28]; lower 2 bits are always 00 due to 4-byte alignment.

Branching Far Away

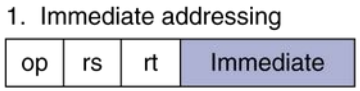
- If branch target is too far to encode with 16-bit offset, assembler rewrites the code
- Example

```
            beq $s0,$s1, L1
                  ↓
            bne $s0,$s1, L2
            j L1
L2:         …
```

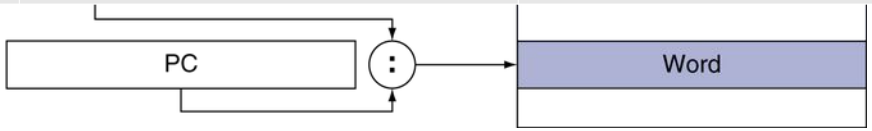
Addressing Mode Summary



Addressing Mode Summary



Addressing Mode	Typical Instructions	Operand / Target	Example
Register	Arithmetic / Logic	Register	add $t0, $t1, $t2
Immediate	Arithmetic with const ant	Constant in instruc tion	addi $t0, $t1, 5
Base / Displaceme nt	Load / Store	Memory address = register + offset	lw $t0, 8($s0)
PC-relative	Conditional Branch	Target = PC + offs et × 4	beq $t0, $t1, Loop
Pseudodirect	Jump	Target from PC + 26-bit address	j Loop



Section 2-5 Quick Questions

Q1. Given memory values and a 32-bit register, determine the results of lb, lbu, and sb independently.

Q2. For a branch instruction at address 1200 with offset 5, determine the PC used, target address, and byte distance.

Q3. Load the given 32-bit constant into $s0 using lui and ori. Determine both immediate values.

Q4. For the Loop example, identify each instruction format and its applicable immediate, shamt, branch offset, or jump address field.

Q5. Given a 32-bit jump target, identify the 26-bit address field and explain why the upper 4 and lower 2 bits are not stored.

Exercise 1 | Byte Memory Operations

Given: $s0 = 0x1000
Memory[0x1000] = 1110 0010 Memory[0x1001] = 0011 0101
$t0 = 10101010 11001100 11110000 00001111
Determine independently:
(a) lb $t1, 0($s0)
(b) lbu $t1, 0($s0)
(c) sb $t0, 1($s0)

Answer

(a) $t1 = 11111111 11111111 11111111 11100010
(b) $t1 = 00000000 00000000 00000000 11100010
(c) Memory[0x1001] = 00001111

Key Point: lb uses sign extension, lbu uses zero extension, and sb stores only the rightmost 8 bits.

Exercise 2 | Branch Addressing and Encoding

A MIPS bne instruction is at address 2400 and branches to target 2320.

(a) Calculate the 16-bit branch offset.
(b) Is the offset positive or negative? Explain briefly.
(c) For bne $t0, $s1, Target, identify op, rs, rt, constant/address.

Answer

PC used = 2400 + 4 = 2404
offset = (2320 − 2404) / 4 = −84 / 4 = −21
(a) offset = −21
(b) Negative; the target is before the branch instruction.
(c) op = bne, rs = $t0, rt = $s1, constant/address = −21

Key Point: A branch offset is measured from PC + 4 in instruction words; a backward branch therefore uses a negative offset.

Exercise 3 | Can the Jump Reach the Target?

A MIPS j instruction is located at:
PC = 0010 1111 1111 1111 1111 1111 1111 0000
Target A = 0010 0000 0000 0000 0000 0001 0000 0000
Target B = 0011 0000 0000 0000 0000 0001 0000 0000

(a) Which target can be directly represented?
(b) Why can the other target not be represented?
(c) For the valid target, identify the 26-bit address field.

Answer

PC[31:28] = 0010; Target A[31:28] = 0010; Target B[31:28] = 0011

(a) Target A

(b) Target B cannot be represented because its upper 4 bits differ from PC[31:28]

(c) 26-bit address field = 00000000000000000001000000

Key Point: A pseudodirect jump reuses PC[31:28], so a direct j target must have the same upper 4 bits as the current PC.